

Your grade: 100%

Your latest: 100% • Your highest: 100% • To pass you need at least 80%. We keep your highest score.

1. In a Processing sketch using Python mode, you are tasked with creating a vector field where the direction and magnitude of each vector are determined by Perlin noise, resulting in a smoothly varying field. You decide to use two separate Perlin noise values to independently set the X and Y components of each vector in the field. Assuming you have a grid of dimensions gridWidth by gridHeight that spans the entire canvas, how can you initialize this vector field using Perlin noise to ensure a natural and organic variation in vector directions and magnitudes?

1 / 1 point

```
1 for i in range(gridWidth):
2     for j in range(gridHeight):
3         noiseVal = noise(i * 0.1, j * 0.1)
4         vectorField[i][j] = PVector(noiseVal, noiseVal)
5
```

```
1 for i in range(gridWidth):
2     for j in range(gridHeight):
3         xComponent = noise(i * 0.1, j * 0.1)
4         yComponent = noise((i + gridWidth) * 0.1, (j + gridHeight) * 0.1)
5         vectorField[i][j] = PVector(xComponent, yComponent)
6
```

```
1 for i in range(gridWidth):
2     for j in range(gridHeight):
3         vectorField[i][j] = PVector.random2D()
4
```

```
1 for i in range(gridWidth):
2     for j in range(gridHeight):
3         xComponent = noise(i * 10, j * 10)
4         yComponent = noise((i + gridWidth) * 10, (j + gridHeight) * 10)
5         vectorField[i][j] = PVector(xComponent, yComponent)
6
```

```
1 noiseValX = noise(0.1)
2 noiseValY = noise(0.2)
3 for i in range(gridWidth):
4     for j in range(gridHeight):
5         vectorField[i][j] = PVector(noiseValX, noiseValY)
6
```

Correct
Correct. This method smartly uses Perlin noise values from different parts of the noise space to generate the X and Y components, ensuring that each vector's direction and magnitude vary smoothly across the field. By offsetting the inputs for the Y component by adding gridWidth and gridHeight, it guarantees that the noise values for the X and Y components are independent, leading to a rich variation in vector directions.

2. In a grid-based system, each cell (or vector) can have up to eight neighbors: north, northeast, east, southeast, south, southwest, west, and northwest. Given a cell at a specific grid position (x, y), what would be the list of tuples that correctly represents the relative positions of all eight neighboring cells, including diagonals?

1 / 1 point

```
1 neighbors = [(0, -1), (0, 1), (-1, 0), (1, 0)]
2
```

```
1 neighbors = [(0, -1), (1, -1), (1, 0), (1, 1), (0, 1), (-1, 1), (-1, 0), (-1, -1)]
2
```

```
1 neighbors = [(1, -1), (1, 1), (-1, 1), (-1, -1)]
2
```

```
1 neighbors = [(2, -2), (2, 2), (-2, 2), (-2, -2), (0, -2), (0, 2), (-2, 0), (2, 0)]
2
```

```
1 neighbors = [(1, 0), (0, -1), (-1, -1), (-1, 0), (-1, 1), (0, 1), (1, 1), (1, -1)]
2
```

Correct
Correct. This list accurately represents the relative positions of all eight neighbors surrounding a central cell in a grid. It includes both the cardinal directions and the diagonals, correctly capturing the full neighborhood of a cell.

3. You are tasked with writing a function in a grid-based system that generates a list of tuples representing the positions of neighboring cells around a central cell located at (0, 0). This function should be flexible, allowing for the specification of how many layers of surrounding neighbors to include. For example, if 1 layer is specified, it should return the immediate eight neighbors; if 2 layers are specified, it should include all cells within two steps of the central cell, and so on. How would you design this function to accommodate any specified number of surrounding neighbor layers?

1 / 1 point

```
1 def getNeighbors(layers):
2     if layers == 1:
3         return [(0, -1), (1, -1), (1, 0), (1, 1), (0, 1), (-1, 1), (-1, 0), (-1, -1)]
4     elif layers == 2:
5         # Manually list out neighbors for 2 layers
6         # ...
7         # And so on for more layers
8
```

```
1 def getNeighbors(layers):
2     neighbors = []
3     for dx in range(-layers, layers + 1):
4         for dy in range(-layers, layers + 1):
5             if dx == 0 and dy == 0:
6                 continue # Skip the central cell itself
7             neighbors.append((dx, dy))
8     return neighbors
9
```

```
1 def getNeighbors(layers):
2     neighbors = []
3     for dx in range(-layers, layers + 1):
4         for dy in range(-layers, layers + 1):
5             neighbors.append((dx, dy))
6     return neighbors
7
```

```
1 def getNeighbors(layers):
2     neighbors = []
3     for dx in range(-layers, layers + 1):
4         for dy in range(-layers, layers + 1):
5             if dx == 0 and dy == 0:
6                 continue
7             neighbors += (dx, dy)
8     return neighbors
9
```

```
1 def getNeighbors(layers):
2     neighbors = []
3     for dx in range(-layers, layers + 1):
4         for dy in range(-layers, layers + 1):
5             if dx == 0 and dy == 0:
6                 continue
7             neighbors[len(neighbors)] = (dx, dy)
8     return neighbors
9
```

Correct
Correct. This solution uses nested loops to generate a list of tuples for the neighbors dynamically, based on the specified number of layers. By iterating from -layers to layers in both dimensions and skipping the central cell (0, 0), it correctly identifies all surrounding cells within the specified range, making it flexible and scalable for any number of layers.